Algoritmo parallelo di Smith-Waterman per allineamento di sequenze DNA

Christian Pozzoli 29678A

Ottobre 2025



Indice

1 Introduzione **3**
1.1 Allineamento di sequenze di DNA 3
1.2 Obiettivo del progetto 3
1.3 Strumenti e ambiente di sviluppo 4

2 Algoritmo **4**
2.1 Algoritmo di Smith-Waterman 4
2.2 Implementazione sequenziale 5

3 Su GPU **7**
3.1 Parallelizzazione dell'algoritmo 7
3.2 Implementazione su GPU 8
3.3 Ottimizzazione con blocchi 9
3.4 Ottimizzazione con memoria condivisa (SMEM) 10
3.5 Ottimizzazione con warp e coalescenza 11
3.6 Analisi delle prestazioni e comparazione 14

1 Introduzione

1.1 Allineamento di sequenze di DNA

L'allineamento di sequenze di DNA è una delle operazioni fondamentali della bioinformatica e rappresenta uno strumento essenziale per lo studio comparativo dei genomi. L'obiettivo principale dell'allineamento è confrontare due o più sequenze di basi di nucleotidi al fine di individuare delle regioni di somiglianza che possono riflettere relazioni evolutive, funzionali o strutturali tra gli organismi.

In termini pratici, un allineamento consiste nel disporre le sequenze una sopra l'altra, introducendo eventuali spazi vuoti ("gap") per massimizzare il numero di corrispondenze tra le basi. Questa operazione consente di identificare mutazioni, inserzioni o delezioni, e di dedurre l'origine comune di geni o proteine. Queste analisi sono fondamentali in vari ambiti applicativi, tra cui

- l'annotazione di nuovi genomi
- la ricerca di omologie tra geni
- la previsione della funzione di proteine sconosciute
- lo studio delle malattie genetiche

Esistono due principali categorie di allineamento: globale e locale. L'allineamento globale, proposto da Needleman e Wunsch (1970), confronta le sequenze nella loro interezza, ed è più adatto quando le sequenze hanno lunghezze simili e un alto grado di somiglianza. L'allineamento locale, introdotto da Smith e Waterman (1981), mira invece a trovare le regioni più simili all'interno di sequenze potenzialmente molto diverse, risultando particolarmente utile in contesti biologici reali, dove solo alcune porzioni delle sequenze mostrano omologia.

1.2 Obiettivo del progetto

L'obiettivo del progetto è implementare e ottimizzare l'algoritmo di Smith-Waterman sfruttando la potenza di calcolo parallelo delle GPU attraverso la piattaforma CUDA. L'algoritmo di Smith-Waterman presenta una complessità computazionale di tipo quadratico rispetto alla lunghezza delle sequenze da confrontare. Questo lo rende poco adatto all'elaborazione di grandi quantità di dati, come quelli generati dalle moderne tecniche di sequenziamento del DNA.

L'obiettivo principale è quindi ridurre i tempi di esecuzione attraverso una parallelizzazione efficiente delle sue operazioni e mantenendo inalterata la precisione dell'algoritmo.

Il progetto mira a dimostrare come l'uso del calcolo parallelo possa rendere praticabile l'utilizzo dell'algoritmo di Smith-Waterman anche in contesti reali e su dataset di grandi dimensioni.

1.3 Strumenti e ambiente di sviluppo

Lo sviluppo del progetto è stato condotto utilizzando il linguaggio CUDA C, sfruttando l'integrazione diretta con l'architettura di calcolo parallelo delle GPU NVIDIA.

L'ambiente di sviluppo è stato configurato su sistema operativo Windows, utilizzando il compilatore `nvcc` in versione 12.9.86, incluso nel CUDA Toolkit 13.0. Il sistema era dotato dei driver NVIDIA in versione 581.57, pienamente compatibili con la versione del toolkit utilizzato.

Per la fase di test e analisi delle prestazioni è stata impiegata una GPU NVIDIA GeForce RTX 3070 (Compute Capability 8.6).

2 Algoritmo

L'algoritmo di Smith-Waterman permette di ottenere un allineamento locale ottimale tramite un approccio di programmazione dinamica. In questa sezione viene descritto il funzionamento dell'algoritmo, la sua formulazione e la relativa implementazione sequenziale, che farà da riferimento per la successiva versione parallela su GPU.

2.1 Algoritmo di Smith-Waterman

L'algoritmo di Smith-Waterman consente di ottenere un allineamento locale ottimale tra due sequenze, identificando le regioni di massima somiglianza. Nell'allineamento vengono considerati tre tipi di operazioni: corrispondenze o sostituzioni, inserzioni e delezioni. Le inserzioni e le delezioni introducono dei "gap" che consentono di allineare correttamente regioni non perfettamente coincidenti tra le sequenze.

L'algoritmo utilizza la tecnica di programmazione dinamica per costruire la matrice di sostituzione in modo da assegnare un punteggio in base ai possibili allineamenti delle sequenze considerate. La soluzione viene poi ricostruita partendo dall'allineamento con il punteggio più alto.

Si articola in quattro fasi principali:

- **Definizione del modello di punteggio e penalità**
  Ogni coppia di basi (o amminoacidi, nel caso di sequenze proteiche) riceve un punteggio in base alla corrispondenza: le corrispondenze esatte vengono premiate con valori positivi, mentre le sostituzioni ottengono punteggi inferiori o negativi. La scelta dello schema di penalità dipende dal tipo di sequenza analizzata e dagli obiettivi dello studio.

- **Inizializzazione della matrice dei punteggi**
  Si costruisce una matrice bidimensionale di dimensioni pari a $(m + 1) \times (n + 1)$, dove m e n rappresentano le lunghezze delle due sequenze. Tutti gli elementi della prima riga e della prima colonna vengono inizializzati a zero. Questa scelta consente all'allineamento di iniziare in qualsiasi posizione delle sequenze e garantisce che i gap iniziali non siano penalizzati.

- **Calcolo dei punteggi (fase di riempimento della matrice)**

$$H_{i,j} = \max \begin{Bmatrix} 0 \\ H_{i-1,j-1} + S(a_i, b_j) \\ H_{i-1,j} + G \\ H_{i,j-1} + G \end{Bmatrix}$$

  Per ogni cella si considerano tre possibili punteggi:

  - il punteggio diagonale (corrispondente a match/mismatch),
  - il punteggio proveniente da sinistra (introduzione di un gap nella sequenza verticale),
  - il punteggio proveniente dall'alto (introduzione di un gap nella sequenza orizzontale).

  Il valore della cella viene impostato come il massimo tra questi tre valori e zero, così da eliminare i contributi negativi e limitare l'allineamento a regioni localmente simili.

- **Traceback (ricostruzione dell'allineamento)**
  Una volta completato il riempimento della matrice, si identifica la cella con il punteggio massimo. A partire da essa, si effettua un traceback seguendo la direzione dei punteggi che hanno contribuito al valore corrente (diagonale, sinistra o alto), fino a raggiungere una cella con valore zero. Il percorso così ottenuto rappresenta l'allineamento locale ottimale. È inoltre possibile individuare allineamenti secondari ripetendo il processo di traceback a partire dai punteggi massimi rimanenti.

Grazie a questa formulazione l'algoritmo di Smith-Waterman garantisce la massima accuratezza possibile nell'allineamento locale. Tuttavia, la sua complessità computazionale quadratica in funzione della lunghezza delle sequenze ($O(m \cdot n)$) ne limita l'applicabilità diretta su dataset di grandi dimensioni, rendendo necessarie strategie di ottimizzazione e parallelizzazione.

2.2 Implementazione sequenziale

```
1  void cpu_alignment(
2                  sw_mat* h,
3                  string seq_1,
4                  string seq_2,
5                  int seq_1_length,
6                  int seq_2_length
7                  ) {
8      for(int i = 1; i < seq_1_length; ++i) {
9          for (int j = 1; j < seq_2_length; ++j) {
10             bool match = seq_1[i] == seq_2[j];
11             int d = h[INDEX(i - 1, j - 1, seq_2_length)].value +
12                             (match ? MATCH : MISMATCH);
13             int u = h[INDEX(i - 1, j, seq_2_length)].value + GAP;
14             int l = h[INDEX(i, j - 1, seq_2_length)].value + GAP;
15             int result = max(max(0, d), max(u, l));
```

```
16
17              sw_mat* h_current = &h[INDEX(i, j, seq_2_length)];
18              h_current->value = result;
19
20              if (result == 0)    h_current->dir = STOP;
21              else if (result == d)   h_current->dir = DIAG;
22              else if (result == u)   h_current->dir = UP;
23              else if (result == l)   h_current->dir = LEFT;
24          }
25      }
26  }
```

Per questa implementazione sequenziale sono stati scelti i seguenti punteggi e penalità secondo basi empiriche e di pratica comune:

- corrispondenza 3
- differenza -3
- gap -2

La matrice è implementata in un array monodimensionale, per garantire coerenza e coesione con i successivi algoritmi paralleli, e l'accesso avviene attraverso una macro, definita di seguito, che linearizza l'indice a partire da riga, colonna e lunghezza della sequenza disposta lungo le colonne.

```
1  #define INDEX(row, col, stride) ( (row) * (stride) + (col) )
```

Le due sequenze sono interscambiabili, in quanto l'obiettivo è quello di trovare degli allineamenti locali, quindi ambivalenti, ma nel codice si è scelto, per motivi di test e chiarezza, di rifersi alle sequenze con nomi quali `main_sequence` e `sequence` (oppure come `sequence_1` e `sequence_2`), mantenendo sempre la `main_sequence` (o `sequence_1`, solitamente la stringa più lunga) un carattere per riga e la `sequence` (o `sequence_2`) un carattere per colonna.

La matrice è, quindi, strutturata come array di `struct sw_mat` definita come struttura ausiliaria composta da un elemento numerico che rappresenta il punteggio dell'elemento e un enumerazione che rappresenta la direzione da cui deriva il punteggio, cioè `DIAG, UP, LEFT` come la direzione dell'elemento che ha fornito la combinazione di punteggio massima, oppure `STOP` ad indicare una casella vuota. La scansione e la popolazione della matrice avviene da sinistra verso destra e dall'alto verso il basso e ogni cella dipende dal valore precedente sulla stessa riga, precedente sulla stessa colonna e precedente sulla stessa diagonale, in modo da valutare la combinazione di punteggio maggiore.

Non verrà discussa in maniera approfondita la logica di traceback, in quanto non è parte dell'argomento di studio. L'algoritmo che ricostruisce la soluzione, come discusso nei capitoli precedenti, parte dal valore maggiore della matrice e ricostruisce la soluzione seguendo il campo `dir` della struct di cui è composta la matrice fino al primo `STOP`.

Per valutare la correttezza degli algoritmi paralleli, però, non viene presa in considerazione la soluzione, ma la matrice stessa per le seguenti ragioni: è più semplice e diretto valutare

l'uguaglianza di due matrici; non sempre la soluzione è unica; è possibile arrivare alla stessa soluzione apparente (sequenza di basi) seguendo due percorsi diversi. In questo contesto la correttezza è determinata dagli stessi valori e dallo stesso percorso delle soluzioni, quindi dalla stessa matrice.

3 Su GPU

3.1 Parallelizzazione dell'algoritmo

L'algoritmo di Smith-Waterman, nella sua formulazione classica (come discusso nel Capitolo 2), si basa sulla programmazione dinamica per calcolare il punteggio di allineamento $H_{i,j}$ per ogni cella della matrice H. La natura della programmazione dinamica introduce una dipendenza rigorosa nel calcolo di $H_{i,j}$: il valore di ogni cella dipende dai valori calcolati nelle celle adiacenti precedentemente completate (in alto, a sinistra e in diagonale).

Questa interdipendenza impedisce il calcolo parallelo dell'intera matrice H, rendendo l'algoritmo naturalmente sequenziale nella sua implementazione. Per sfruttare le architetture parallele, è necessario identificare e strutturare la computazione in modo da massimizzare l'indipendenza e il parallelismo.

Esistono due approcci principali per affrontare questa dipendenza e parallelizzare l'algoritmo:

- **Indipendenza antidiagonale:** Osservando la matrice, si nota che tutte le celle situate sulla stessa antidiagonale dipendono unicamente dalle celle calcolate sulle antidiagonali precedenti. Quindi, tutte le celle su una singola antidiagonale possono essere calcolate in parallelo.

- **Indipendenza della singola colonna:** Questo approccio tenta di parallelizzare il calcolo degli elementi all'interno di una singola colonna (o riga). Sebbene ogni cella $[i, j]$ dipenda intrinsecamente da $[i-1, j]$ (elemento immediatamente sopra) e $[i, j-1]$ (elemento immediatamente a sinistra), l'analisi dell'algoritmo di Smith-Waterman con l'uso di ottimizzazioni può permettere di ristrutturare il calcolo per minimizzare o eliminare la dipendenza verticale tra le righe i e $i-1$ per una colonna fissa j. L'obiettivo è calcolare tutti gli elementi della colonna j in parallelo, una volta che la colonna $j-1$ è stata completamente calcolata.

Nel contesto di questo progetto di calcolo su GPU, si è scelto di adottare la strategia basata sull'indipendenza della singola colonna. Questa scelta è motivata da diversi fattori pratici e computazionali:

- **Massimizzazione del parallelismo per colonna:** Assegnare un thread (o un blocco di thread) alla computazione di tutti gli elementi di una colonna offre un elevato e uniforme grado di parallelismo, specialmente per sequenze lunghe, semplificando la gestione del carico di lavoro e la mappatura tra thread e dati.

- **Sfruttamento della coalescenza:** L'accesso ai dati lungo una colonna è fondamentale per le prestazioni su GPU. Mantenere l'accesso ai dati in modo sequenziale favorisce la coalescenza degli accessi alla memoria globale.

- **Semplicità di sincronizzazione:** L'intera operazione si riduce a n passaggi sequenziali (il numero di colonne). Tra un passaggio e l'altro è necessaria una barriera di sincronizzazione, rendendo la logica di controllo più semplice rispetto alla complessa gestione delle sincronizzazioni tra richiesta dall'approccio antidiagonale puro.

L'approccio della soluzione si basa sull'iterazione seriale sulle colonne della matrice e il calcolo parallelo degli elementi lungo la colonna. L'elemento a sinistra ($H_{i,j-1}$) è già disponibile nella matrice risultante dal lancio del kernel precedente, così come l'elemento sulla diagonale ($H_{i-1,j-1}$), poiché disponibili nella colonna precedente. La cella superiore, invece, è la dipendenza più critica. Per eliminarla e consentire il calcolo in parallelo lungo la colonna, si fa affidamento sul concetto di gap e sul fatto che il valore stesso della cella superiore sia necessariamente determinato da dei gap verticali a partire da un elemento diagonale o iniziale, in quanto il valore sulla colonna attuale deriva da una combinazione di spostamenti verticali e diagonali. Il calcolo si riconduce a una catena di dipendenze verso l'alto e prosegue verso l'alto fino a quando si raggiunge un elemento $H_{k,j}$ la cui dipendenza non è verso l'alto, ma verso la cella diagonale (ovvero, c'è un match tra i caratteri), oppure si raggiunge l'inizio della matrice [1]. Questo permette di ricavare il valore da una cella $H_{k-1,j-1}$, della colonna precedente, purché $S_1[k] = S_2[j]$ sia il primo match trovato salendo da i a 1:

$$U = H_{k-1,j-1} + (i - k) \cdot \text{GAP} + \text{MATCH}$$

Questo calcolo, pur essendo più intensivo in termini di istruzioni, elimina la dipendenza verticale tra $H_{i,j}$ e $H_{i-1,j}$, rendendo tutti i calcoli di lungo la colonna indipendenti gli uni dagli altri.

3.2 Implementazione su GPU

La computazione della matrice di sostituzione sfrutta la tecnica descritta di parallelizzazione sulle righe. Il loop esterno avanza colonna per colonna e ad ogni iterazione il thread con ID TID viene mappato all'elemento $H_{i,j}$ con $i = TID + 1$. Tutti i thread calcolano simultaneamente i loro rispettivi $H_{i,j}$. Alla fine del singolo ciclo è necessaria una sincronizzazione per garantire che tutti i thread abbiano completato prima di procedere alla colonna successiva.

```
__global__ void gpu_alignment(sw_mat* h,
                              char* seq_1,
                              char* seq_2,
                              int seq_1_length,
                              int seq_2_length)
{
    int i = threadIdx.x + 1;
    for (int j = 1; j < seq_2_length; ++j) {
        bool match = seq_1[i] == seq_2[j];
        int d = h[INDEX(i - 1, j - 1, seq_2_length)].value +
```

```
                    (match ? MATCH : MISMATCH);
        int l = h[INDEX(i, j - 1, seq_2_length)].value +
                    GAP;
        int u = 0;

        bool stop = false;
        for(int k = i - 1; k > 0 && !stop; --k) {
            if(seq_1[k] == seq_2[j]) {
                u = h[INDEX(k - 1, j - 1, seq_2_length)].value +
                    (i - k) * GAP + MATCH;
                stop = true;
            }
        }

        int result = max(max(0, d), max(u, l));

        sw_mat* h_current = &h[INDEX(i, j, seq_2_length)];
        h_current->value = result;

        if (result == 0)    h_current->dir = STOP;
        else if (result == d)   h_current->dir = DIAG;
        else if (result == u)   h_current->dir = UP;
        else if (result == l)   h_current->dir = LEFT;

        __syncthreads();
    }
}
```

Questo approccio avviene su un singolo blocco e ha degli evidenti limiti dettati dalla grandezza dell'input da elaborare, in quanto prevede che l'intera stringa di basi sia minore o uguale a 1024 caratteri, cioè il limite di thread per un singolo blocco.

Ai fini di testare l'algoritmo, quest'ultimo è stato applicato a una sequenza di basi di 4500 caratteri allineandola con una sequenza principale di 1024 caratteri, ottenendo uno speedup di media 3.5 volte rispetto alla versione sequenziale.

```
    Output:
        CPU time: 0.0695081s
        GPU time: 0.019001s
        Speed up: 3.65813
```

3.3 Ottimizzazione con blocchi

Per superare il limite del singolo blocco è possibile dividere la computazione della singola colonna in più blocchi e arrivare così a coprire un input che supera i soli 1024 caratteri. Per farlo, però, è necessario sincronizzare i blocchi in modo che procedano alla colonna successiva soltanto dopo che tutti i thread di tutti i blocchi abbiano finito la di processare la colonna. Per garantire questo comportamento si divide, quindi, la computazione tra host e device in modo che l'host gestisca la sincronizzazione tra colonne e il device esegua il kernel sulla singola colonna.

La chiamata a livello host sarà la seguente

```
    for(int j = 1; j < cols; ++j) {
        gpu_alignment<<<grid, block>>>(
                                    d_h1,
                                    d_main_seq,
                                    sequence[j],
                                    rows,
                                    cols,
                                    j);
        cudaDeviceSynchronize();
    }
```

L'istruzione `cudaDeviceSynchronize()` funge da barriera di sincronizzazione tra una colonna e quella successiva, anche se è possibile e vantaggioso sfuttare la sincronizzazione implicita. Infatti, togliere la sincronizzazione esplicita non permette ai kernel di sovrapporsi poiché sono implicitamente eseguiti sullo stesso stream di default (non avendone definito nessuno) e vengono automaticamente sincronizzati in modo che possano essere eseguiti in maniera sequenziale. Inoltre, questo tipo di sincronizzazione permette di ottenere dei vantaggi a livello di scheduling automatico e gestione delle risorse, che porta a una maggiore efficienza di esecuzione e tempistiche migliori.

Con queste modifiche, quindi, è possibile estendere le dimensioni dell'input e ottenere uno speedup maggiore. Su un input di 4500 caratteri per entrambe le sequenze è possibile ottenere uno speedup in media di 7.5 volte superiore alla versione sequenziale e 2.2 volte la versione con la sincronizzazione esplicita.

```
    Output:
        |=============RESULTS=============
        | CPU time:               0.2919s
        | GPU time:               0.0845s
        | GPU time (stream-sync): 0.0375s
        |============SPEED UP=============
        | Over CPU:               3.45383
        | Over CPU (stream-sync): 7.78143
        |==================================
        | Over GPU (stream-sync): 2.25299
        |==================================
```

3.4 Ottimizzazione con memoria condivisa (SMEM)

L'algoritmo ora soffre di un collo di bottiglia intrinseco legato alla latenza dell'accesso alla Global Memory, che è notevolmente superiore a quella dei registri o della Shared Memory (SMEM). In particolare, per calcolare ogni cella $H_{i,j}$ della colonna corrente j, sono necessari i tre valori provenienti dalla colonna precedente $j-1$: $H_{i,j-1}$ (sinistra), $H_{i-1,j-1}$ (diagonale) e il valore $H_{k-1,j-1}$ utilizzato nel calcolo della versione parallela.

Per mitigare la latenza causata dai continui accessi alla Global Memory per recuperare i dati della colonna $j-1$, si è adottata una strategia di caching tramite la Memoria Condivisa (SMEM).

La Shared Memory è una risorsa accessibile esclusivamente ai thread all'interno dello stesso blocco. Sfruttando questa proprietà, l'ottimizzazione è come segue: i dati più fre-

quentemente riutilizzati sono i punteggi e le direzioni della colonna $j-1$ immediatamente adiacente a quella corrente j e per ogni blocco di thread responsabile di una porzione della colonna j, si alloca una regione di Shared Memory, dimensionata per contenere i valori $H_{i,j-1}$ e $H_{i-1,j-1}$ (sinistra e diagonale). Poiché il calcolo di $H_{i,j}$ dipende da $H_{i,j-1}$ (sinistra) e $H_{i-1,j-1}$ (diagonale), e un blocco di thread gestisce un intervallo di righe i da $i_{\min}$ a $i_{\max}$, è necessario caricare i valori $H_{i,j-1}$ per l'intervallo $[i_{\min}-1, i_{\max}]$. All'inizio dell'esecuzione del kernel per la colonna j, ogni thread carica il proprio valore $H_{i,j-1}$ (necessario per il calcolo dal valore a sinistra) dalla Global Memory nella SMEM locale. Un thread specifico all'inizio del blocco deve caricare anche il valore $H_{i_{\min}-1,j-1}$ (necessario come elemento diagonale $H_{i-1,j-1}$ per il primo thread del blocco).

Un punto di sincronizzazione (`__syncthreads()`) è obbligatorio dopo la fase di caricamento per assicurare che tutti i dati della colonna $j-1$ richiesti dal blocco siano stati resi visibili nella SMEM a tutti i thread prima che inizino i calcoli di $H_{i,j}$.

Una volta completato il caricamento, l'elemento $H_{i,j}$ viene calcolato recuperando i valori $H_{i,j-1}$ e $H_{i-1,j-1}$ direttamente dalla Memoria Condivisa a bassa latenza, anche se alcuni valori nel calcolo del valore superiore potrebbero attingere alla Memoria Globale in quanto non sono presenti tutti i dati della colonna precedente inficiando sulle performance generali. Questa tecnica non altera l'algoritmo di base, ma lo rende più efficiente, riducendo significativamente il numero di transazioni off-chip e la latenza d'accesso.

Su una prova con input di 4500 caratteri per entrambe le sequenze l'accesso alla memoria globale è avvenuto per 61400 caratteri su circa 2035000 accessi totali, quindi lo 0.3% delle volte. Lo speedup medio è di 1.2 volte la versione senza Shared Memory.

```
Output:
    |=============RESULTS=============
    | CPU time:              0.299836s
    | GPU time (no SMEM):    0.047507s
    | GPU time (SMEM):       0.038001s
    |============SPEED UP=============
    | Over CPU (no SMEM):    6.3114
    | Over CPU (SMEM):       7.8902
    |==================================
    | Speed up SMEM:         1.25015
    |==================================
```

3.5 Ottimizzazione con warp e coalescenza

Per massimizzare le prestazioni del calcolo su GPU, non è sufficiente sfruttare la memoria a bassa latenza (SMEM); è essenziale ottimizzare il throughput della memoria e l'esecuzione a livello dell'unità base, cioè il warp. L'architettura hardware delle GPU è progettata per operare in modo efficiente quando i thread all'interno dello stesso warp accedono a indirizzi di Memoria Globale contigui (o che ricadono nella stessa linea di cache), ottenendo così la coalescenza degli accessi. La strategia di parallelizzazione per colonna (ogni thread gestisce una riga i per una colonna fissa j) richiede che gli elementi consecutivi lungo l'asse delle righe $(i, i+1, i+2, \dots)$ siano contigui in memoria per sfruttare la coalescenza. Per questo, è stato adottato un layout di memoria column-major, semplicemente modificando la macro di indicizzazione utilizzata per la matrice H:

$$\text{Row major: } \text{INDEX}(i, j, N_{\text{cols}}) = i \cdot N_{\text{cols}} + j$$
$$\text{Column major: } \text{INDEX}(i, j, N_{\text{rows}}) = j \cdot N_{\text{rows}} + i$$

Dove N_{rows} è la lunghezza della prima sequenza (lo *stride*). Grazie a questa modifica, l'incremento dell'indice del thread i corrisponde all'incremento dell'indirizzo di memoria. Ciò garantisce che il caricamento e la scrittura dei risultati della colonna corrente j, e la lettura della colonna precedente $j-1$, siano coalescenti, riducendo il numero di transazioni sulla Memoria Globale e aumentando la banda effettiva.

La performance delle GPU è estremamente sensibile alla divergenza, che si verifica quando i thread all'interno dello stesso warp eseguono istruzioni differenti a causa di branch (`if/else`).

Nel calcolo del punteggio diagonale nell'algoritmo, è necessaria una condizione per discriminare tra un `match` e un `mismatch`. Per eliminare l'istruzione di branching associata a questa condizione, è stata implementata una versione branchless del kernel, sostituendo la logica condizionale con un'espressione aritmetica equivalente:

$$D = H_{i-1,j-1} + ((\text{match} \cdot 3) + ((1 - \text{match}) \cdot -3))$$

Dove `match` è la variabile booleana che è determinata dall'uguaglianza dei caratteri confrontati. Questa tecnica garantisce che tutti i thread eseguano il medesimo percorso di istruzioni, ottenendo un significativo incremento di prestazioni.

Un ulteriore livelli di ottimizzazione prevede l'uso di istruzioni di warp shuffle nel kernel. L'obiettivo è quello di utilizzare questa funzione CUDA a livello warp per accelerare ulteriormente il recupero del valore diagonale $H_{i-1,j-1}$, scambiando il dato tra i registri dei thread adiacenti anziché accedendo alla Memoria Globale. Tramite `__shfl_up_sync`, il thread i tenta di ottenere il valore $H_{i-1,j-1}$ dal registro del thread $i-1$ e, sebbene teoricamente ad altissima efficienza per lo scambio di registri, introduce un leggero overhead dovuto alla sincronizzazione warp implicita e alla gestione della divergenza del thread 0, il quale deve comunque caricare il valore dalla Memoria Globale non avendo a disposizione alcun thread con indice $i-1$ nel suo corrente warp.

L'analisi dei risultati di esecuzione conferma l'efficacia delle ottimizzazioni e dimostra come l'eliminazione del branching aumenti le prestazioni e sia superiore perfino alla versione con il warp shuffle, infatti quest'ultima risulta più lenta della versione *no branch*, per via dell'overhead di sincronizzazione imposto da `__shfl_up_sync` e l'efficienza della cache L1. Data la perfetta coalescenza del layout column-major, l'accesso a $H_{i-1,j-1}$ nella versione no branch è gestito con un costo di latenza molto basso, che risulta complessivamente inferiore all'overhead dello shuffle e della gestione del caso limite del thread 0, che introduce ulteriore divergenza. Sebbene la coalescenza e la programmazione a livello warp siano fondamentali, il maggiore speedup è stato ottenuto garantendo l'assenza di divergenza nel calcolo del punteggio.

Su una prova con input di 4500 caratteri per entrambe le sequenze la versione che usa coalescenza senza branching arriva a uno speedup di 15 volte superiore alla versione sequenziale.

```
Output:
    |=============RESULTS==============
    | CPU time:               0.461732s
    | GPU time (warp):        0.033999s
    | GPU time (no branch):   0.029998s
    | GPU time (shuffle):     0.032508s
    |=============SPEED UP=============
    | Over CPU (warp):        13.5808
    | Over CPU (no branch):   15.3921
    | Over CPU (shuffle):     14.2036
    |==================================
    | Over warp (no branch): 1.13337
    | Over warp (shuffle):   1.04586
    |==================================
```

Parallelamente all'ottimizzazione dei warp, è possibile tentare di ridurre ulteriormente la latenza di accesso alla memoria utilizzando la Memoria Condivisa. L'ipotesi teorica è che, caricando esplicitamente la colonna $j-1$ nella SMEM, si possano servire le richieste dei thread molto più velocemente rispetto all'accesso alla Memoria Globale.

L'implementazione con SMEM prevede che ogni thread del blocco carica il valore corrispondente $H_{i,j-1}$ dalla Memoria Globale alla SMEM, e come descritto in precedenza, una barriera `__syncthreads()` assicura che tutti i dati siano visibili in modo da venire letti dalla SMEM invece che dalla memoria globale.

Questa versione risulta meno efficiente della versione no-branch che accede direttamente alla Memoria Globale. Grazie all'ottimizzazione del layout in column-major, gli accessi alla colonna $j-1$ risultano perfettamente coalescenti. Inoltre, data la forte località temporale (i dati vengono scritti o letti nell'iterazione precedente), questi risiedono quasi certamente nella cache L1 della GPU. Al contrario, l'uso della SMEM impone un costo fisso in termini di istruzioni e, soprattutto, richiede la barriera `__syncthreads()`. Quest'ultima costringe tutti i warp del blocco ad attendersi, interrompendo la pipeline di esecuzione e riducendo la capacità dello scheduler di nascondere la latenza. In conclusione, in uno scenario di accesso perfettamente sequenziale e coalescente, il costo amministrativo della Memoria Condivisa supera il guadagno rispetto a un hit in cache L1, rendendo l'accesso diretto alla Memoria Globale la strategia migliore.

Mantenendo un input di 4500 caratteri per entrambe le sequenze la versione warp-SMEM è stata comparata con la versione warp (senza branching) senza SMEM e la versione base SMEM. La versione coalescente con la SMEM supera la versione base SMEM, ma rimane comunque meno efficiente della versione solamnente coalescente.

```
Output:
    |================RESULTS================
    | CPU time:                     0.291688s
    | GPU time (SMEM):              0.036000s
    | GPU time (warp):              0.029999s
    | GPU time (warp SMEM):         0.030444s
```

```
|================SPEED UP================
| Over CPU (SMEM):              8.10244
| Over CPU (warp):              9.72325
| Over CPU (warp SMEM):         9.58116
|========================================
| Over SMEM (warp):             1.20004
| Over SMEM (warp SMEM):        1.1825
|========================================
| Over warp (warp SMEM):        0.985387
|========================================
```

3.6 Analisi delle prestazioni e comparazione

In questa sezione vengono presentati i risultati sperimentali ottenuti dalle diverse implementazioni dell'algoritmo Smith-Waterman su GPU, confrontandoli con l'esecuzione sequenziale su CPU. L'obiettivo è valutare l'efficacia delle diverse strategie di ottimizzazione introdotte: parallelismo naive, utilizzo della memoria condivisa (SMEM), primitive Warp-Shuffle e l'approccio ibrido Warp-SMEM.

I test sono stati condotti variando due parametri fondamentali: la dimensione dei blocchi CUDA (block size) e la lunghezza delle sequenze di input (N). La metrica principale utilizzata per la valutazione è il tempo di esecuzione, misurato in secondi, da cui è stato derivato lo *speedup* rispetto alla controparte sequenziale.

La configurazione della dimensione del blocco (numero di thread per blocco) è cruciale per massimizzare l'occupazione della GPU e nascondere le latenze di accesso alla memoria. Sono stati eseguiti test con configurazioni di blocco variabili da 32 a 1024 thread, e dimensioni di input variabili.

L'implementazione con coalescenza ha mostrato le prestazioni migliori in assoluto, raggiungendo il minimo tempo di esecuzione con una dimensione del blocco di 512 thread. Le versioni basate su SMEM e Parallelismo Naive hanno mostrato una maggiore sensibilità alla dimensione del blocco, con prestazioni degradanti per blocchi molto piccoli o molto grandi, trovando spesso un ottimo locale intorno ai 128-256 thread. L'uso delle primitive warp permette di ridurre drasticamente l'overhead di sincronizzazione e l'uso della memoria condivisa, rendendo l'algoritmo più robusto rispetto alle variazioni nella configurazione dei blocchi.

Per valutare la scalabilità, le implementazioni sono state testate con sequenze di lunghezza variabile da 5.000 a 20.000 caratteri. I risultati evidenziano un andamento lineare del tempo di esecuzione rispetto alla dimensione dell'input, comportamento atteso per l'algoritmo Smith-Waterman. Tuttavia, la pendenza della curva per la CPU è significativamente più ripida rispetto alle versioni GPU: La CPU passa da 0.43s per $N = 5.000$ a circa 6.98s per $N = 20.000$. La versione warp su GPU scala molto più efficientemente, passando da 0.039s a 0.170s nello stesso intervallo.

La tabella riassuntiva e i grafici di speedup mostrano chiaramente la gerarchia di prestazioni tra le diverse ottimizzazioni. Lo speedup è calcolato come $\frac{T_{CPU}}{T_{GPU}}$.

In conclusione, l'ottimizzazione basata su warp si è rivelata la più efficace, offrendo uno speedup superiore a 40 volte. Questo risultato è dovuto all'accesso coalescente in memoria globale che permette di fornire più dati con singole transazioni di memoria. La versione SMEM offre comunque un notevole miglioramento rispetto all'approccio parallelo base, ma rimane limitata dalla banda della memoria condivisa e dalla necessità di barriere di sincronizzazione esplicite, che sono meno frequenti o assenti nella logica warp.

Riferimenti bibliografici

[1] Bo Chen, Yun Xu, Jiaoyun Yang, and Haitao Jiang. A new parallel method of smith-waterman algorithm on a heterogeneous platform. In Ching-Hsien Hsu, Laurence T. Yang, Jong Hyuk Park, and Sang-Soo Yeo, editors, *Algorithms and Architectures for Parallel Processing*, pages 79–90. Springer Berlin Heidelberg.